



面向服饰纹理替换的智能重绘 方法研究与系统开发

张雅瑞
2026-05-18

目录

CONTENTS

01

绪论

02

需求分析和
功能设计

03

重绘方法和
系统原理

04

重绘系统的
设计与实现

05

总结与展望



01

绪论

背景：

- 围绕在电商和虚拟试衣领域，用户和设计师的服装纹理替换需求

痛点：

- 传统方法效率低
- 现有AI缺陷、处理效果差

课题目标：

开发一套“智能重绘 (iRetexturing)”的算法和框架。

应用价值：

- 为设计师提供一站式原型工具；
- 优化电商用户的购物决策

可行性：

- SDXL、ControlNet和SAM等成熟算法框架
- React和Next.js的系统全栈架构

相关工作：

- 生成对抗网络（GAN）不稳定、存在模式崩溃
- 扩散模型（Diffusion Models）生成效果好、潜力大

本课题创新点之一：

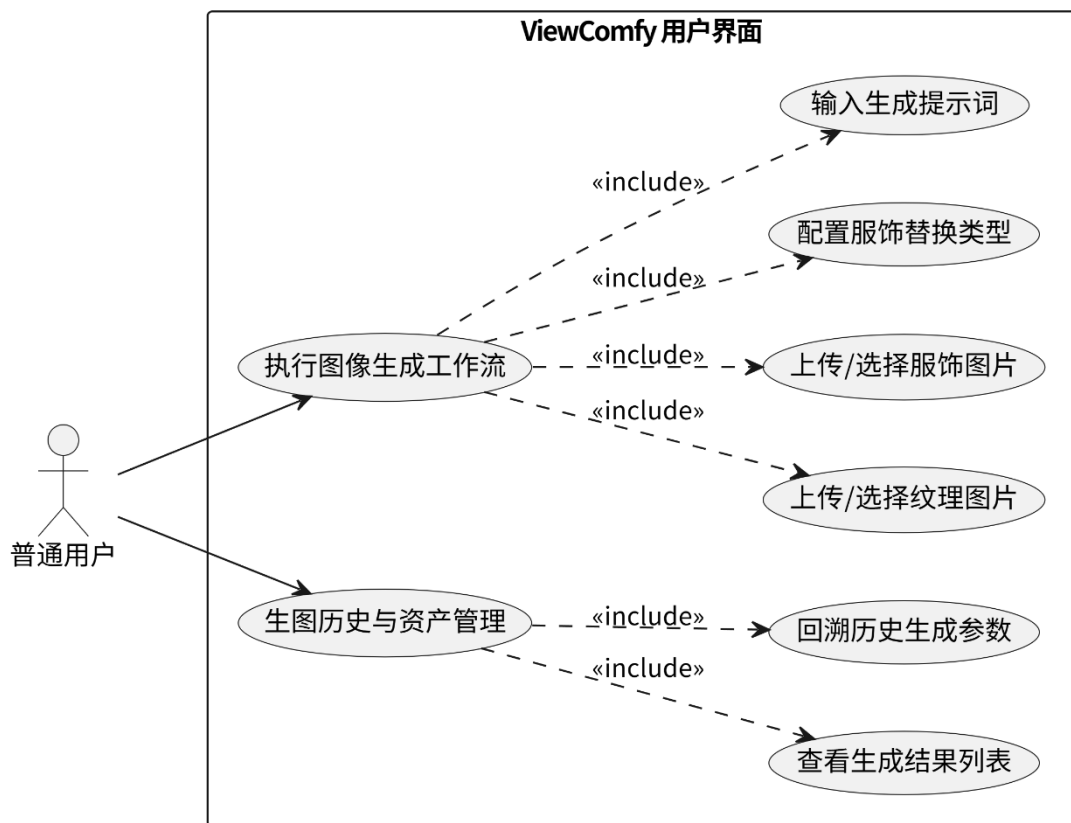
- 将扩散模型引入服装纹理重绘，并利用ControlNet等算法加强生成效果，对比baseline实现SOTA级别效果



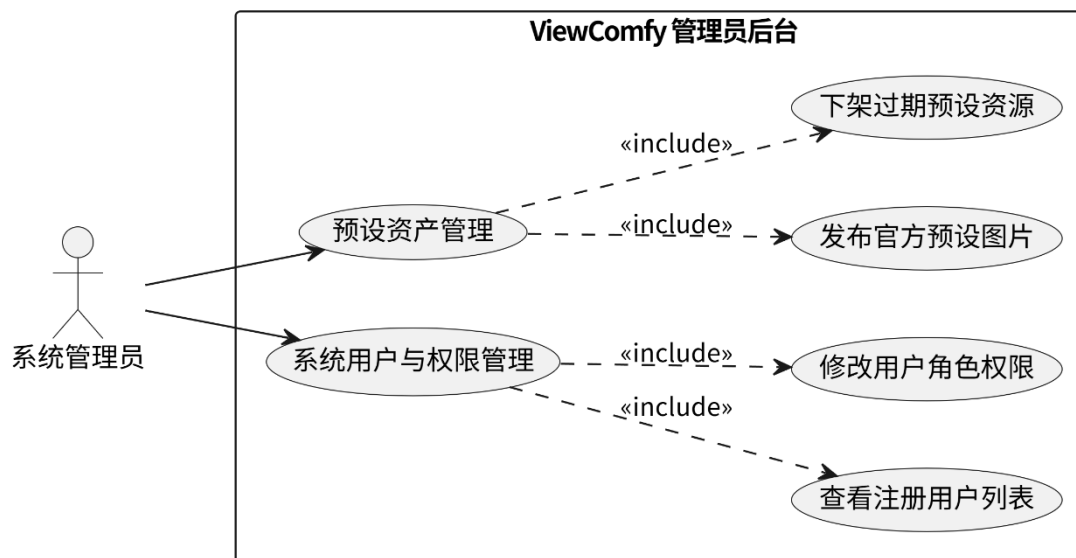
02

需求分析和功能设计

系统需求分析：



用户模块用例图

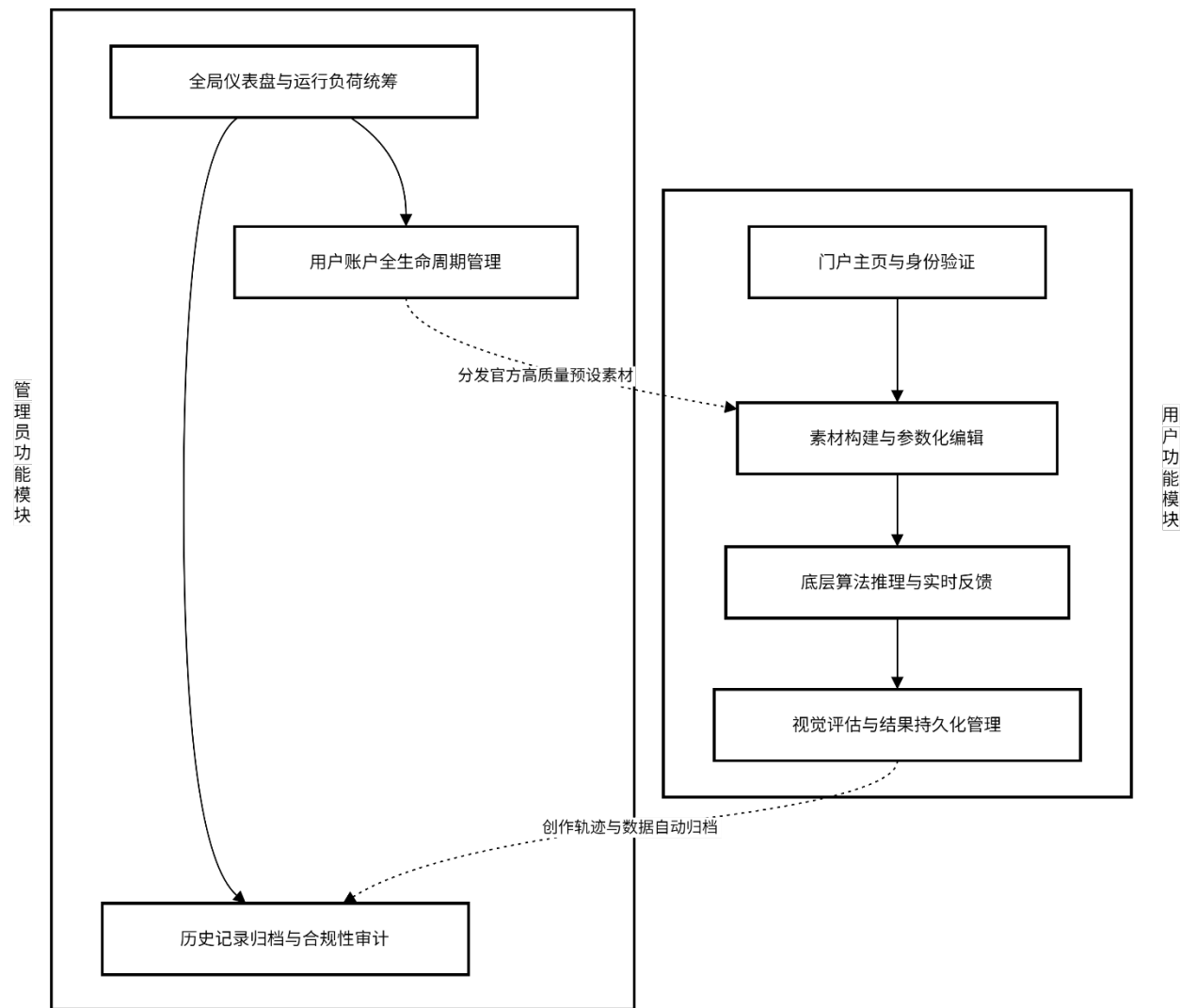


管理员模块用例图

功能需求分析：

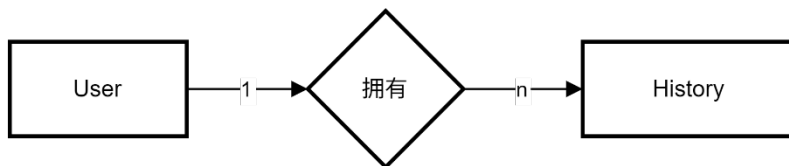
- **低门槛高响应：** 适配消费级低配硬件，轻量化流程
- **专业纹理编辑：** 支持四方连续无缝面料，可精细调节纹理
- **素材高效管理：** 内置预设素材库、支持自定义上传，批量导出
- **运维安全合规：** 后台全量用户管理、操作日志全程留痕

系统设计-功能设计

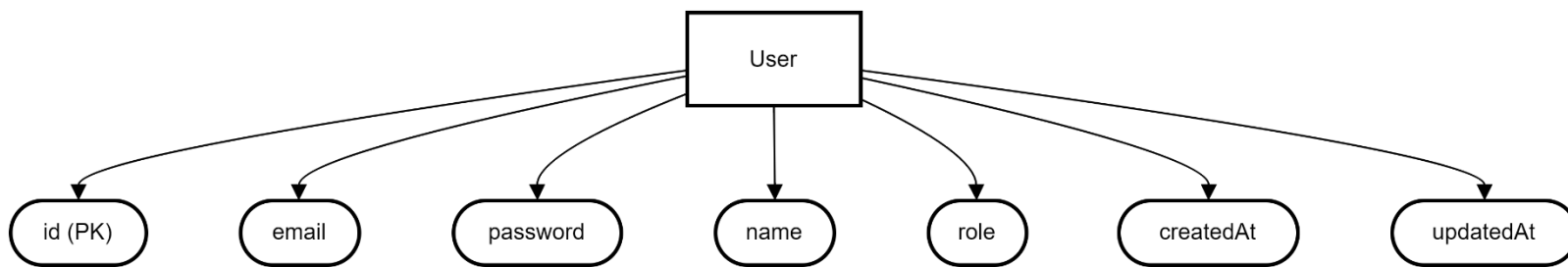


功能设计流程图

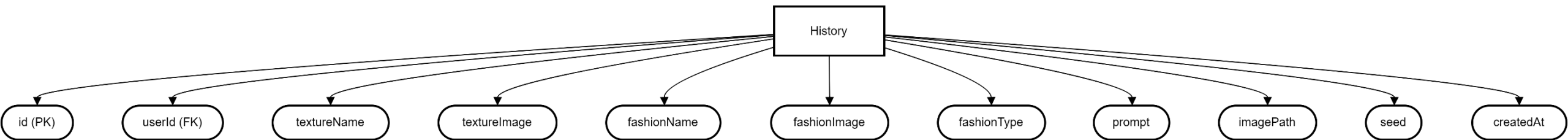
系统设计-数据库设计



整体数据库ER 图



用户数据库ER 图



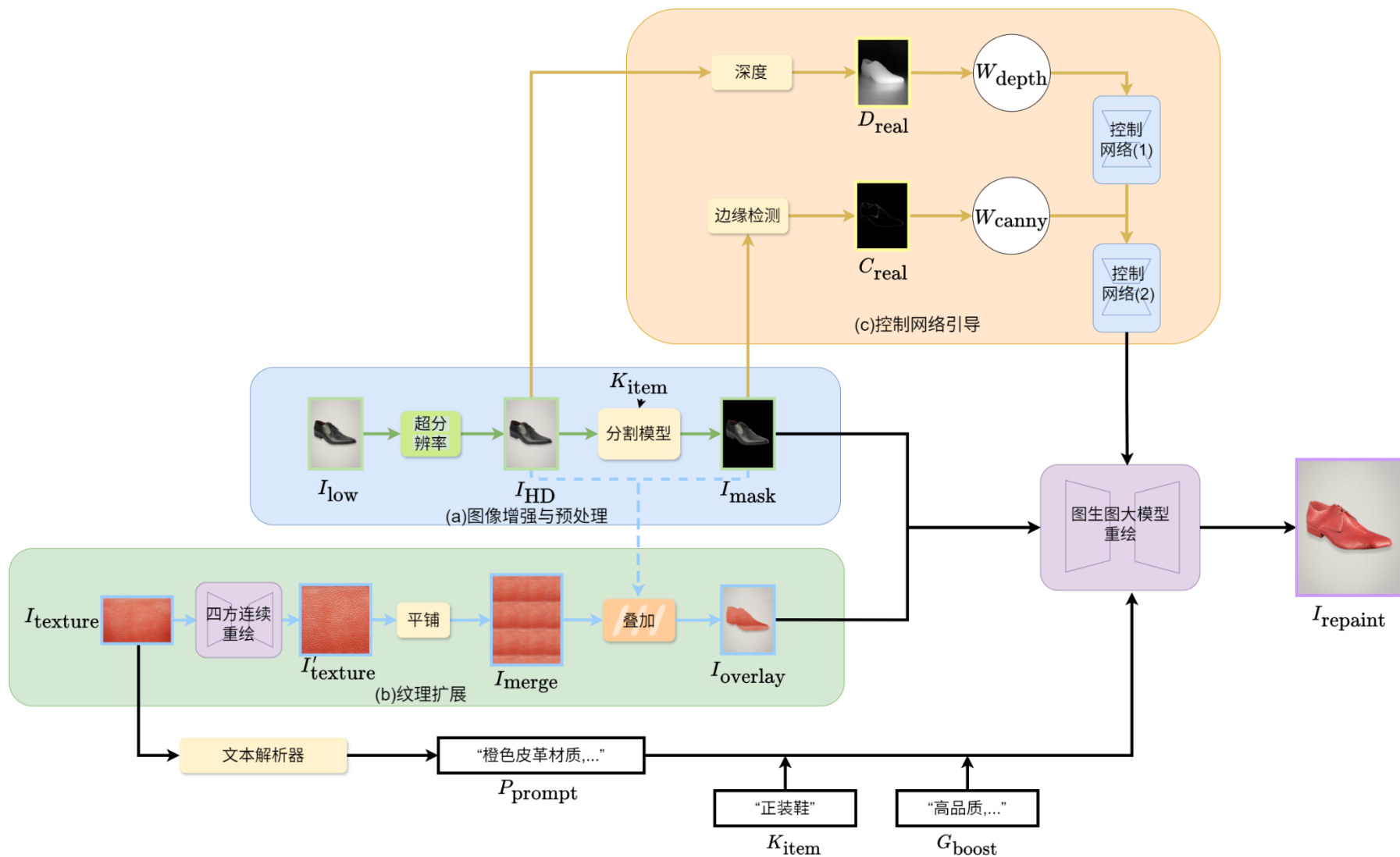
历史记录数据库ER 图



03

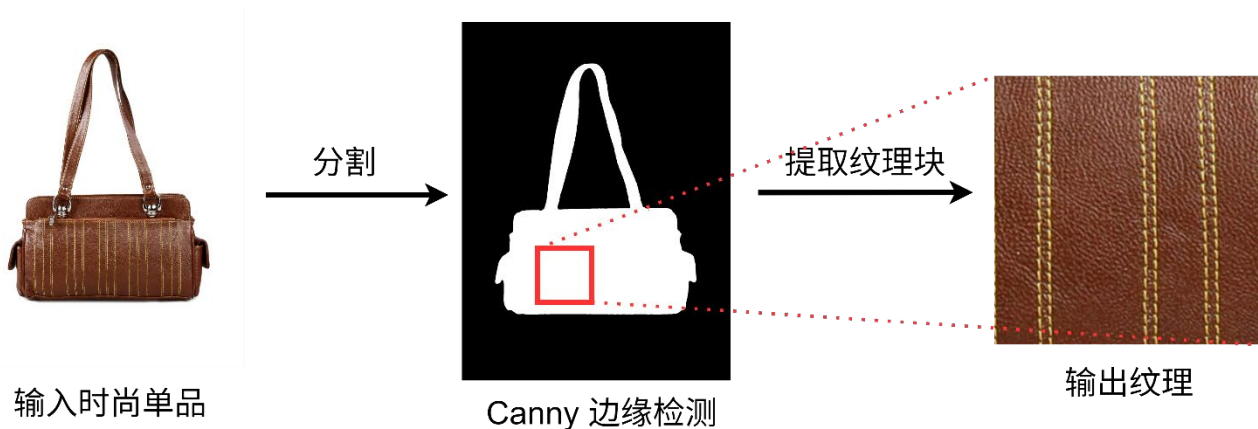
重绘方法和系统原理

算法概述



算法概述

实验设计与分析-数据集处理



手提包的纹理提取过程示例

Input: Fashion item image I

```
mask = SAM_segmentation(I) // Segment the fashion item from
                             the image
mask = convert_to_binary(mask) // Convert mask to boolean
array
rectangle = compute_largest_inscribed_rectangle(mask)
// Find largest rectangle within the mask
cropped_image = crop(I, rectangle) // Crop image to rectangle
bounds
if (cropped_image.size < 256 × 256) then // Check if cropped
    region is too small
    return None
end if
// Exclude if rectangle is too small
texture_patch = random_crop(cropped_image, size=256)
if (not detect_edges(texture_patch, threshold1=100, thresh-
    old2=200)) then
// Apply edge detection to check texture content
return None
end if
// Discard if no edges detected
return texture_patch
// Return the extracted texture patch
```

Output: texture_patch (256 × 256 texture patch, or None if extraction fails)

伪代码



Method	LPIPS	SSIM	FID
Texturize ⁸	0.5847	0.1204	277.5
Texture-Reformer ⁹	0.6161	0.3792	274.9
CP-VTON ¹⁰	0.1452	0.8043	97.91
Leffa ¹¹	0.2383	0.7432	99.42
DiffFashion ²¹	0.3034	0.5690	260.8
DiffuseIT ⁴¹	0.1618	0.8074	54.96
Our method	0.1385	0.8323	75.25

表 3-2 不同方法的对比结果

Method	Time/100 images (s)	Resolution	Pixels/image (MP)	Time/million pixels (s/MP)	Hardware
Texturize ⁸	18.93	300 × 400	0.12	1.58	RTX3060
CP-VTON ¹⁰	30.66	192 × 256	0.049	6.26	RTX3060
Texture-Reformer ⁹	69.05	448 × 600	0.269	2.57	RTX3060
DiffuseIT ⁴¹	2539.99	192 × 256	0.049	518.37	RTX3060
Leffa ¹¹	2335.17	900 × 1000	0.9	25.95	RTX4090
DiffFashion ²¹	4412.74	256 × 256	0.065	672.0	RTX3060
Ours (RTX4090)	3494.30	768 × 1024	0.786	44.43	RTX4090
Ours (RTX3060)	12,885.30	768 × 1024	0.786	163.75	RTX3060

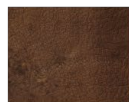
表 3-3 不同硬件下的图片处理过程的对比

实验设计与分析-对比实验

输入

时尚单品

纹理



Texturize



Texture-Reformer

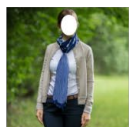


CP-VTON



输出

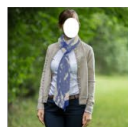
Leffa



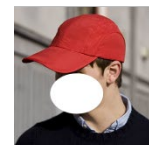
DiffFashion



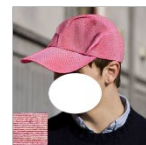
DiffuseIT



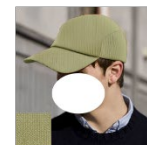
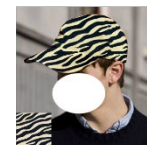
Ours



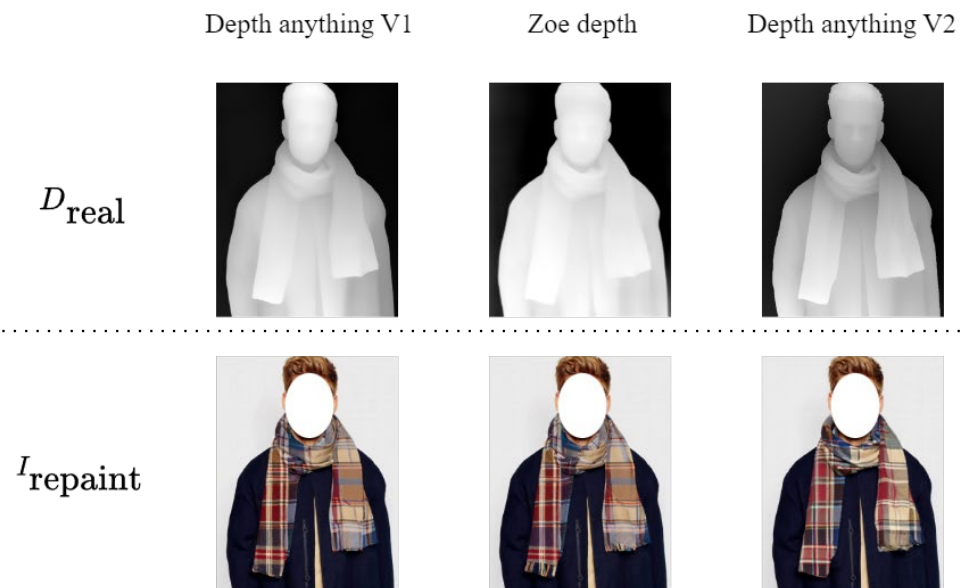
原始图像



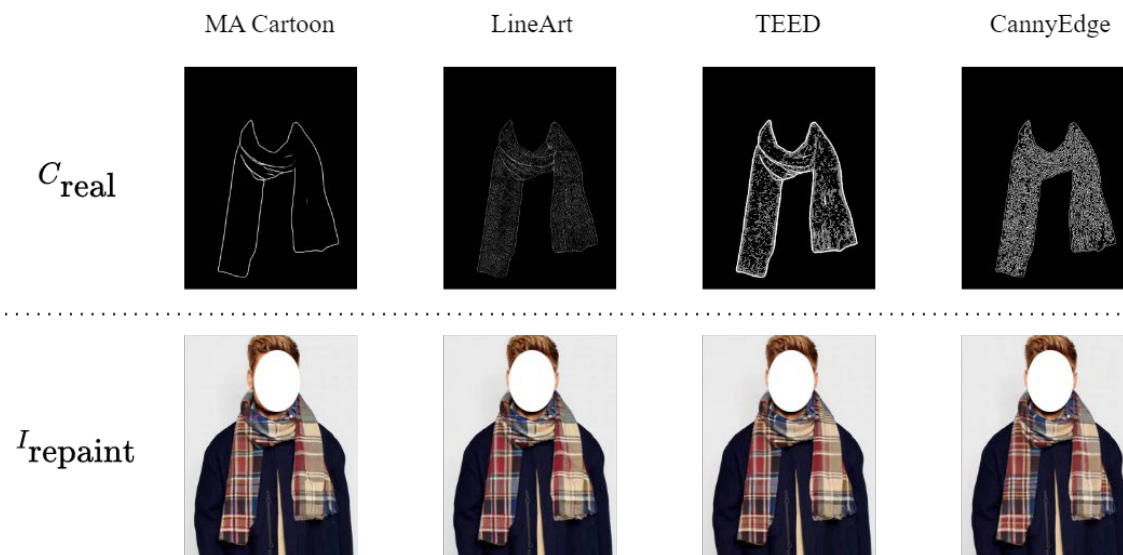
重绘纹理图像



可视化结果

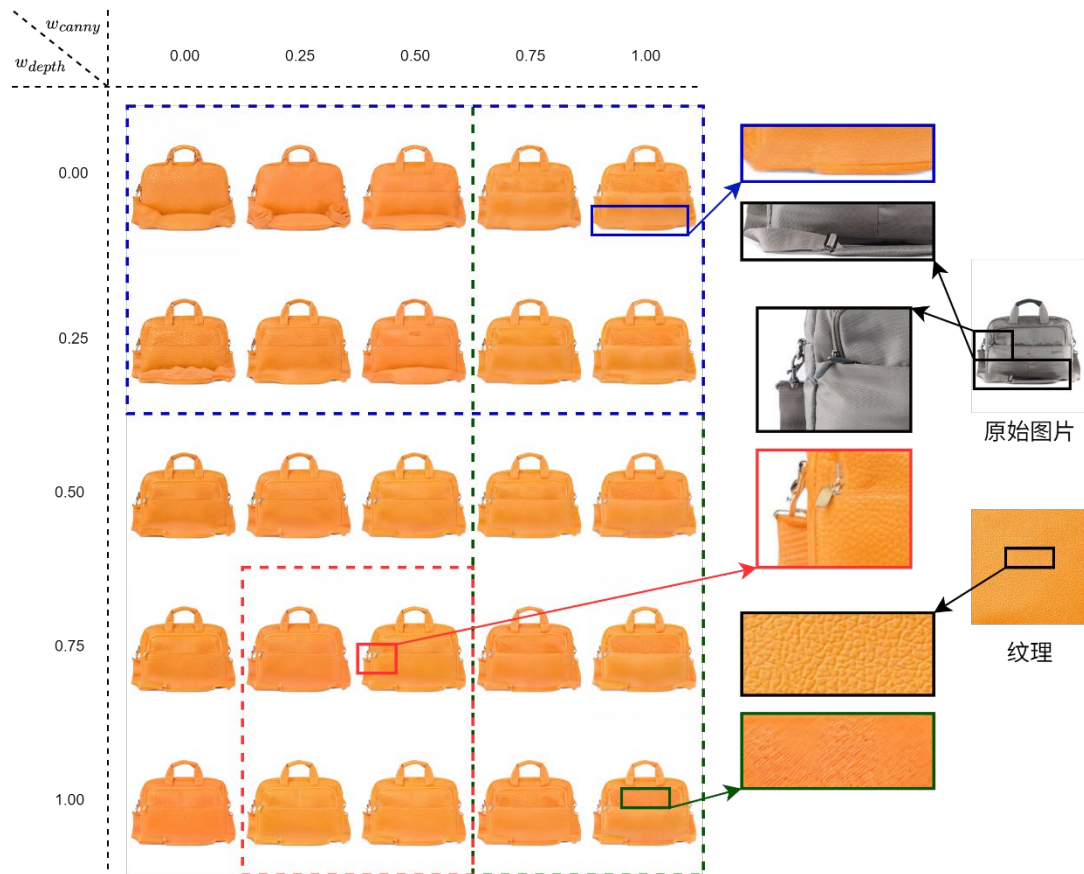


深度估计算法评估

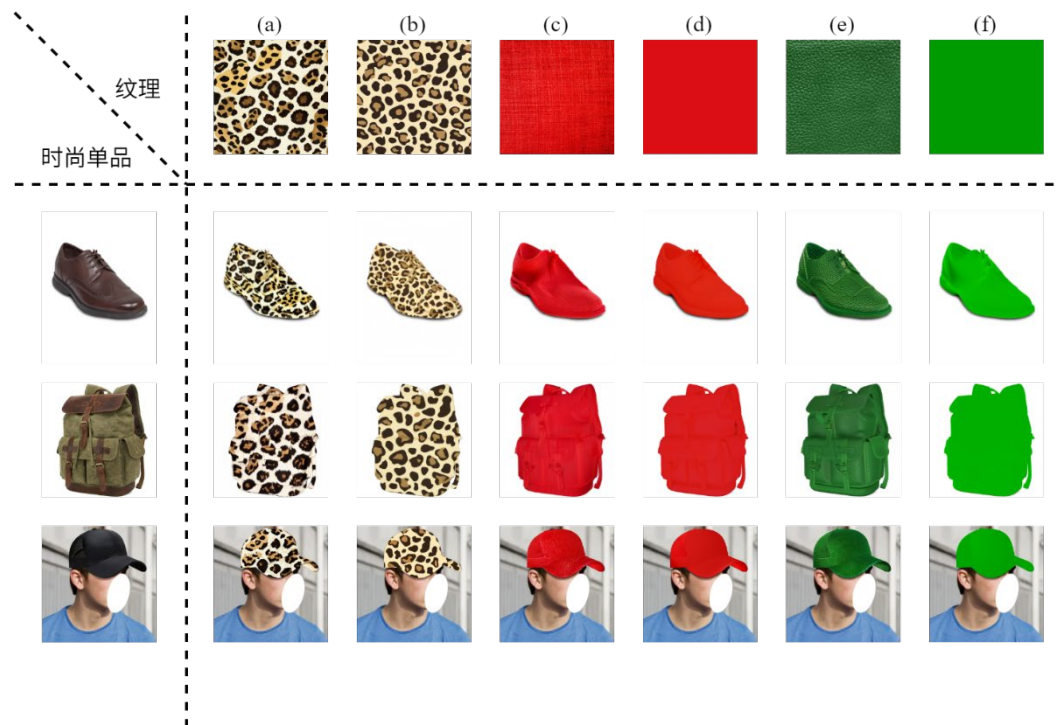


线稿边缘检测算法评估

实验设计与分析-消融实验

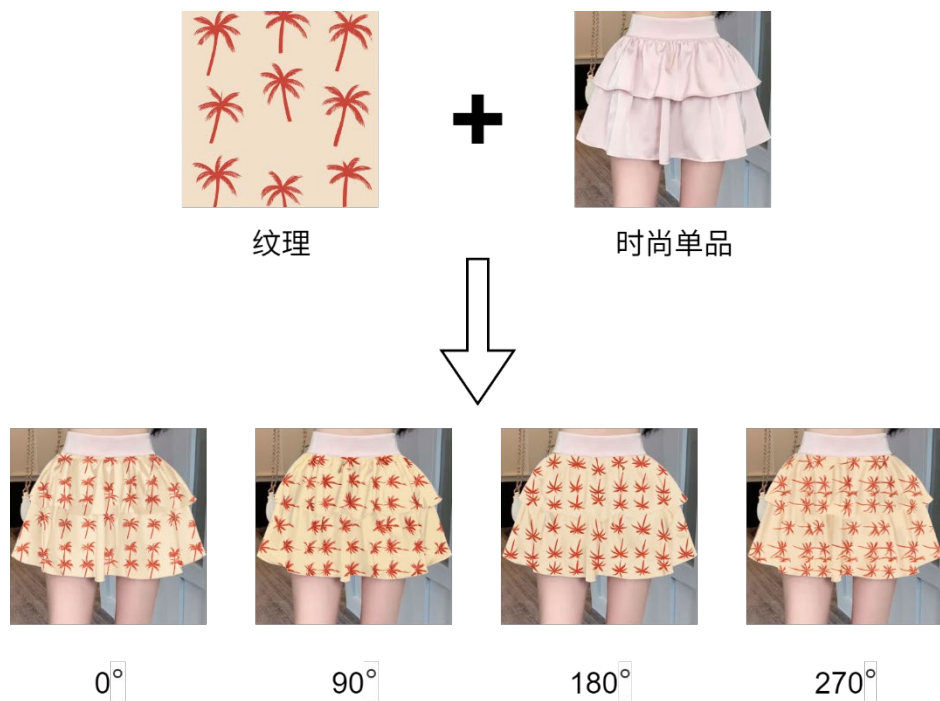


参数敏感性分析



纹理细节对重绘效果的影响机制

实验设计与分析-消融实验



多角度纹理映射



平铺数量及缩放比例对重绘效果的影响



04

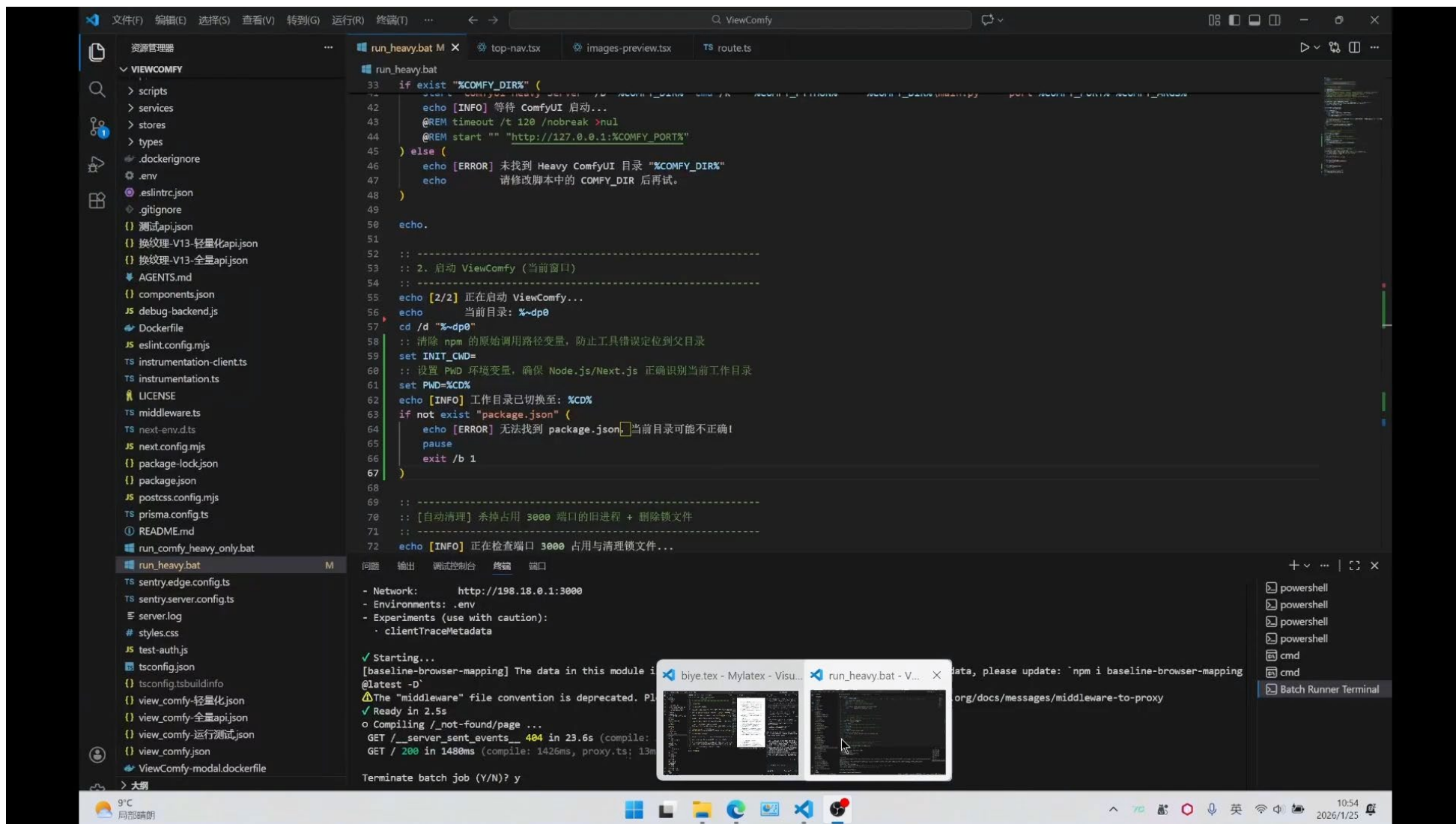
重绘系统的设计与实现

重绘系统的设计与实现

系统特征:

- Next.js 全栈架构与 TypeScript 类型共享
- 响应式编辑界面与高保真图像预览交互
- API Routes 微服务编排与 JWT/RBAC 安全体系
- ComfyUI 推理接入与 WebSocket 实时进度反馈
- Prisma/SQLite 数据持久化与可追溯审计管理

系统演示





05

总结与展望

总结与展望

主要工作内容：

- 在系统落地层面。本课题围绕服饰纹理替换任务，搭建了一套靠 Next.js 和 ComfyUI 配合工作的端到端智能重绘平台。
- 在算法研究层面，设计出并且落地了融合多模态空间约束与四分连续性生成策略的智能重绘方法。

未来改进方向：

- 微观纹理保真度弱
- 平坦服饰失立体感
- 复杂形变几何失真



感谢您的观看